Complexity:-

* Asymptotic Notations:-

Big-Oh (O)

Suppose, $f(n) = 5n+4$ → Upper Bound (Max Time)

$g(n)$

$f(n) = O(g(n))$	$f(n) \leq c \times g(n)$ for $n \geq n_0$
$5n+4 = O(n)$	$5n+4 \leq c \times n$ // $c=9$

where c is constant

$3n^3 + 20n^2 + 5 \leq (28) n^3$

only add we constants from this eq.

7n - 2 is O(n)

where c > 0 and
$n_0 \geq 1$

where n_0 is the smallest value of n for which the inequality is true.

When we say f(n) is O(g(n)), we mean that as n becomes large enough (greater than or equal to n_0), the function f(n) doesn't grow faster than a constant multiple of g(n)

1) O(1) Constant // ****
2) O(n) Linear // ***
3) $O(n^2)$ Quadratic // nested loops // Bad
4) O(log(n)) // Good as compare to Linear
5) O(n log(n)) // Good as compare to Quadratic
6) $O(2^n)$ // Recursion // Too Bad

if $f(n) = 3n^3 + 2n^2 + 5$

then $g(n) = n^3$

$O(n^3)$

$g(n)$ describes the growth rate.

Big-Omega

↳ Lower Bound
(Min. time)

Omega	Ω
Theta	Θ
Oh	O

$f(n) = \Omega(g(n))$

$c > 0 \quad n_0 \geq 1$

$f(n) \geq c \times g(n)$ for $n \geq n_0$

Practice

* fun()

{

a = b ; — 1

b = c ; — 1

c = d ; — 1

d = c + b + c ; — 3

return d; — 1

}

(7) => O(1)

Because of Constant

* for (int i=1; i<=n; i++) — 1, (n+1), n

{ stat; } — n

=> 3n + 2

=> O(n)

* for(int i=1; i<=n;) — 1

{ s; }

```
h = 1
while (h <= n)
{
    S;
    h = 2*h;
}
```

1) h = 1
2) h = 2
3) h = 4
4) h = 8
5) h = 16

$h = 2^0, 2^1, 2^2, 2^3, 2^4, \ldots, 2^k$

where 2^k is the first power of 2 that exceeds or equal to n.

$2^k \geq n$

Take log base 2 both side.

$\log_2 2^k \geq \log_2 n$

$k \geq \log_2 n$

loop runs $\log_2 n + 1$ times approx.

$O(\log_2 n)$

* for(int i=1; i<2; i++) — ① ② (n-1)

{ S1; } — (n-1)

* for(int i=1; i<n; i+=2) — ① (n/2)+1 (n/2)

{ S1; } — (n/2)

$$1 + \frac{n}{2} + 1 + \frac{n}{2} + \frac{n}{2}$$

$$\Rightarrow \frac{3n}{2} \Rightarrow O(n)$$

* for (int i=1; i<=n; i++)
  for (int j=1; j<=n; j++)
    { S; }

1 + (n+1) + n = 2n+2 ⇒ O(n)
n * n ⇒ O(n²)

* for(int i=1; i<=n; i++)
  for (int j=1; j<=m; j++)
    { S; }

1 + (n+1) + n = 2n+2 ⇒ O(n)
n * m ⇒ O(m)

2mn + 3n + 1 ⇒ O(m×n)

* for (int i=1; i<=n; i++)
  {
    for(int j=1; j<=i; j++)
      { S; }
  }

1 + (n+1) + n = 2n+2 ⇒ O(n)
n(n+1)/2 ⇒ O(n²)

$3\left(\frac{n(n+1)}{2}\right) + 3n + 1$

$O(n^2)$

i = 1 j: 1

i = 2 j: 1+2

i = 3 j: 1+2+3

i = 4 j: 1+2+3+4

⋮

i = n j: 1, 2, 3, 4 ... n

↳ $\frac{n(n+1)}{2}$

for n = 5 ⇒ $\frac{5(5+1)}{2}$

⇒ (15) ⇒ 1+2+3+4+5

① ①+1 n

for (int i = 1; i ≤ n; i++)

for (int j = 1; j ≤ n; j++)

{ S; }

Suppose, $n=5$

$i=1$	$j = 1,2,3,4$	$(n-1)$
$i=2$	$j = 1,2,3$	$(n-2)$
$i=3$	$j = 1,2$	$(n-3)$
$i=4$	$j = 1$	$(n-4)$

Total iterations for $j =$

$$(n-1)+(n-2)+(n-3)+(n-4)\cdots+1+0$$

$$\therefore \text{Sum} = \frac{n(n+1)}{2}$$

$$\frac{(n-1)((n-1)+1)}{2} = \frac{(n-1)*n}{2}$$

$$O(n^2)$$

This is an arithmetic Series as last term is 0, Starts from $(n-1)$ to last Term as 0.

* ~~for (int i=1; i<n; i++)~~
~~{ int j=1; j<~~

* for(int i=1; i<n; i=i*2)
{ S1; }

↳ See third Example.

i=1	1 time	2^0
i=2	2 times	2^1
i=3	4 times	2^2
i=4	8 times	2^3
⋮		
i=n	2^{n-1} times	
log(n)	$2^{\log(n-1)}$	
log(n)+1	$2^{\log(n)}$	

↳ $n \log_2(2)$

```
for(int i=n; i>=1; i=i/3)
    { S; }
```

Suppose n = 21

i = n

Steps	i	i vals
1)	$n/3^0$	// 21
2)	$n/3^1$	// 7 // 21/3
3)	$n/3^2$	// 2 // 7/3 // 21/9
4)	$n/3^3$	// 0 // 2/3 // 21/27
5)	$n/3^4$	

$$\frac{n}{3^k} \leq 1$$

equality sign change when n goes otherside

$$n \geq 3^k$$

$$\log_3 n \geq \log_3 3^k$$

$$k \leq \log_3 n$$

$$k = \log_3 n$$

$$O(\log n)$$

* for(int i=1; i<=n; i++)
{ for(int j=1; j<=n; j=j+2)
{ S1; }
}

$j = 2^k \Rightarrow 2^k \le n$

$k <= \log_2 n$

$n \times \log_2 n$

$O(n \log n)$

* for (int i=1; i<=n; i=i*3)
{ for (int j=1; j<=n; j=j+2)
{ S1; }
}

$\log_3(n) \times n/2$

$O(n \log n)$

1D Array

6

* for(int i=1; i<=n; i=i*2)
  for (int j=1; j<i; j++)

i=1 j=1

i=2 j=1*2

i=8 (i=4) j=1*2, 3, 4, 5, 6, 7, 8

j=1, 2, 3, 4

$T(n) = 1 + 2 + 4 + 8 + \cdots 2^k$

$T(n) = 2^{k+1} - 1$

$\approx 2^{\log(n)+1} - 1 = 2n - 1$

$O(n)$

To get address of an Element

1D :-

A[i] = Base address + (i * size of element)

2D :-

A[i][j] = BA + (i * Y + j) * size of element

BA = Base Address
X = Number of Rows
Y = Number of Columns

3D :-

A[i][j][k] = BA + (i * Y * Z + j * Z + k) * size of elem

X = Number of Depth
Y = Number of Rows
Z = Number of Cols

To get Address of Column

2D:

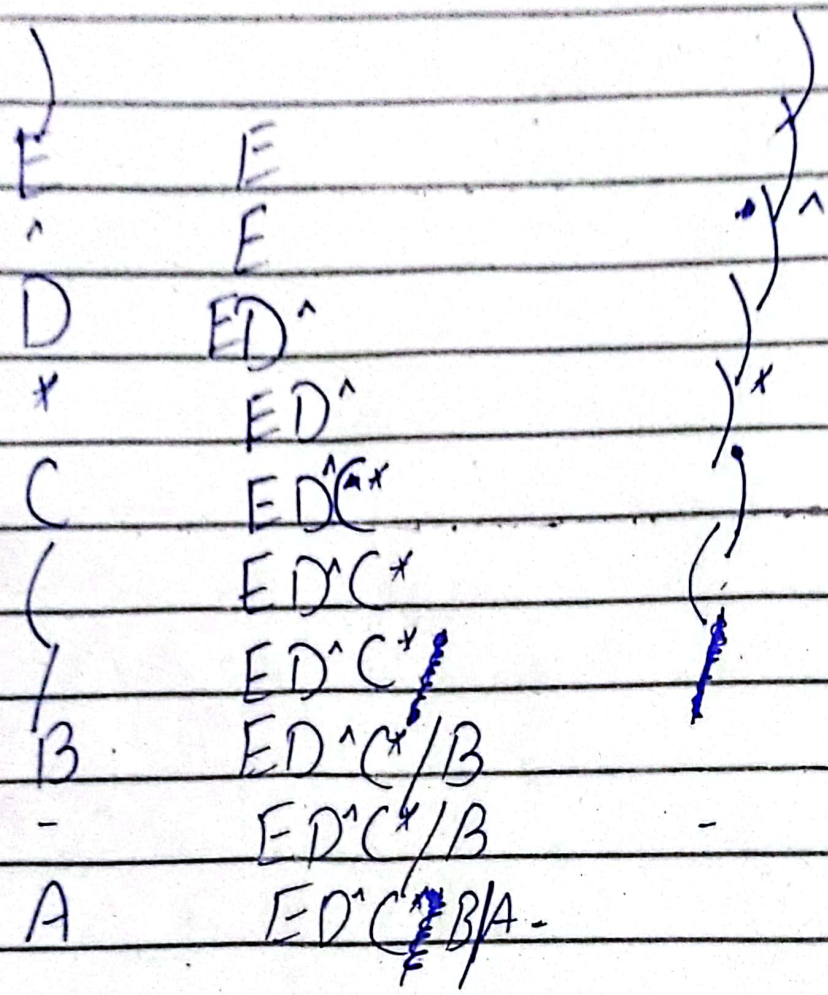
Base Address + (Column Index * Size of element)

3D:

Base Address + (Column Index * depth * Size of Element)

AB/CDEF/GH+/+-*

)E^D*C(/B-A



-A/B*C^DE

AB / C*D - GH +

2

A - B / (C * D ^ E)

A	A	
-	A	-
B	AB	-
/	AB	- /
(	AB	- / (
C	ABC	- / (
*	ABC	- / (*
D	ABCD	- / (*
^	ABCD	- / (* ^
E	ABCDE^*	- / (* ^
)	ABCDE^*/	
	ABCDE^*/-	

F - + / / AB CD / E F + GH

CDE^*/BA-/

BACDE^*/-

)H+G(/F/E+D-C*B/A

*HG+F/E/DC-B*A/+

+/A*BC-D/E/F/+GH

MNOP

M/N-(O+P)*O-(R+(S+T))

UVW*+XY/-ZAB^-*

$((A/B) * (D-C)) + ((E/F)/(K+H))$

AB/DC-*EF/KH+/+

A/B*D-C+E/F/(K+H)

AB/DC-*EF/KH+/+

(A/B)*(D-C)+(E/F)/(K+H)

AB/DC-*EF/KH+/+

AB/DC-*EF/KH+/+

+*/AB-DC//EF+KH

AB/D*C-EF/KH+/+-

A/B*D-C+E/F/(K+H)

AB/D*C-E+F/KH+/

Symbol	Postfix string	operator
A	A	
/	A	/
B	AB	/
*	AB/	*
D	AB/D	*
-	AB/D*	-
C	AB/D*C	-
+	AB/D*C-	+
E	AB/D*C-E	+
/	AB/D*C-E+	/
F	AB/D*C-E+F	/
/	AB/D*C-E+F/	/
(	AB/D*C-E+F/	/(
K	AB/D*C-E+F/K	/(
+	AB/D*C-E+F/K	/(+
H	AB/D*C-E+F/KH	/(+
)	AB/D*C-E+F/KH+	/
	AB/D*C-E+F/KH+/	

AB/D*C-E+F/KH+/

A	AB
B	AB
/	AB /
D	ABD /
*	A/BD *
C	A/BDC *
-	A/B*DC -
E	A/B*DCE -
+	A/B*D-CE +
F	A/B*D-CEF +
/	A/B*D-CEF +/
K	A/B*D-CEFK +/
H	A/B*D-CEFKH +/
+	A/B*D-CEFKH +
/	A/B*D-C+EFKH +/
	A/B*D-C+E/FKH +/
	A/B*D-C+E/F/(K+H)